0102. Binary Tree Level Order Traversal

m | Category: Trees | LeetCode: https://leetcode.com/problems/binary-tree-level

Problem

- Return node values level by level, left to right.
- Example: [3,9,20,null,null,15,7] → [[3],[9,20],[15,7]].

Intuition

- BFS with a queue visits nodes in level order. Snapshot the queue size at the start of each level to know where it ends.

Brute Force

- DFS passing depth and appending to result[depth] also works (O(n)); BFS is the natural fit.

Optimal Approach

- Queue = [root].
- While queue non-empty: size = queue.size(); pop size nodes into a level list, pushing their children.
- Append the level.

Worked Example

```
queue [3]        -> level [3]      push 9,20
queue [9,20]     -> level [9,20]   push 15,7
queue [15,7]     -> level [15,7]
```

Java Solution

```java
public class BinaryTreeLevelOrderTraversal {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
        while (!q.isEmpty()) {
            List<Integer> level = new ArrayList<>();
            for (int i = q.size(); i > 0; i--) {
                TreeNode n = q.poll();
                level.add(n.val);
                if (n.left != null) q.add(n.left);
                if (n.right != null) q.add(n.right);
            }
            res.add(level);
        }
        return res;
    }
}
```

Complexity

- Time: O(n). Space: O(width) up to O(n).

Pitfalls

- Read the size before the inner loop; the queue grows while you iterate.